

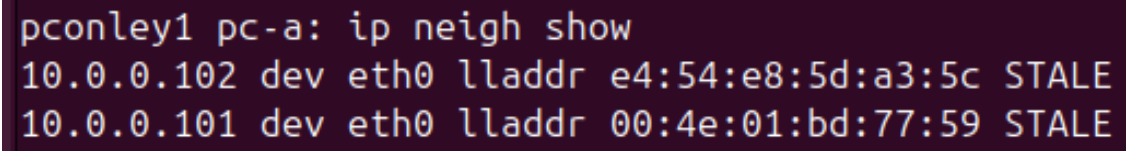
1 Address Resolution Protocol (BYENG 217)

The Address Resolution Protocol (ARP) is a fundamental protocol in computer networks that translates IP addresses to MAC (hardware) addresses. This experiment explores ARP behavior including basic request/response mechanisms, caching, neighbor reachability detection, and the ARP state machine.

1.1 Exercise 1.1: ARP Table

Part 1: Initial ARP Tables

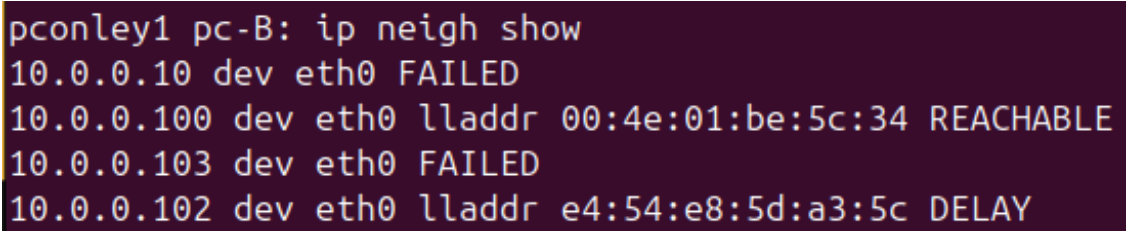
Screenshot 1: PC-A Initial ARP Table



```
pconley1 pc-a: ip neigh show
10.0.0.102 dev eth0 lladdr e4:54:e8:5d:a3:5c STALE
10.0.0.101 dev eth0 lladdr 00:4e:01:bd:77:59 STALE
```

Figure 1: PC-A initial ARP table showing learned neighbor entries

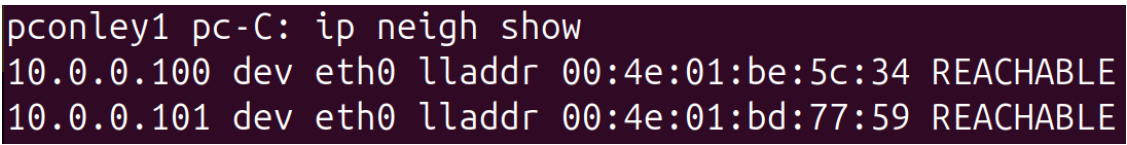
Screenshot 2: PC-B Initial ARP Table



```
pconley1 pc-B: ip neigh show
10.0.0.10 dev eth0 FAILED
10.0.0.100 dev eth0 lladdr 00:4e:01:be:5c:34 REACHABLE
10.0.0.103 dev eth0 FAILED
10.0.0.102 dev eth0 lladdr e4:54:e8:5d:a3:5c DELAY
```

Figure 2: PC-B initial ARP table showing learned neighbor entries

Screenshot 3: PC-C Initial ARP Table



```
pconley1 pc-C: ip neigh show
10.0.0.100 dev eth0 lladdr 00:4e:01:be:5c:34 REACHABLE
10.0.0.101 dev eth0 lladdr 00:4e:01:bd:77:59 REACHABLE
```

Figure 3: PC-C initial ARP table showing learned neighbor entries

Part 2: ARP Tables After Deletion

Screenshot 4: PC-A After ARP Table Flush

```
pconley1 pc-a: ip neigh del 10.0.0.101 dev eth0
pconley1 pc-a: ip neigh del 10.0.0.102 dev eth0
pconley1 pc-a: ip neigh show
pconley1 pc-a:
```

Figure 4: PC-A ARP table after executing `ip neigh flush all`

Screenshot 5: PC-B After ARP Table Flush

```
pconley1 pc-B: ip neigh del 10.0.0.100 dev eth0
pconley1 pc-B: ip neigh del 10.0.0.102 dev eth0
pconley1 pc-B: ip neigh show
10.0.0.10 dev eth0 FAILED
10.0.0.103 dev eth0 FAILED
```

Figure 5: PC-B ARP table after executing `ip neigh flush all`

Screenshot 6: PC-C After ARP Table Flush

```
pconley1 pc-C: ip neigh del 10.0.0.100 dev eth0
pconley1 pc-C: ip neigh del 10.0.0.101 dev eth0
pconley1 pc-C: ip neigh show
pconley1 pc-C:
```

Figure 6: PC-C ARP table after executing `ip neigh flush all`

After deletion, the ARP tables are empty, requiring hosts to perform ARP resolution before any IP communication can occur.

1.2 Exercise 1.2: Basic ARP Request and Response

Part 1: Ping with ARP Required (pc-a-arp.pcap)

Screenshot 1: Head command output showing first ping with ARP

```
pconley1 pc-a: head pc-a-arp.txt
16:22:19.440908 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
16:22:21.441059 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
16:22:23.441161 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
16:22:25.068922 LLDP, length 185: pconley1-switch-a
.....
.Ethernet1...x
.pconley1-switch-a.XArista Networks EOS version 4.29.1FX-710P-DHCP running on an Arista Networks CCS-710P-12.....
.....$...
```

Figure 7: First 10 lines of pc-a-arp.txt showing ARP Request/Reply followed by ICMP Echo Request/Reply

The ARP Request shows PC-A broadcasting "who-has 10.0.0.101" with source MAC 00:4e:01:be:5c:34 (PC-A) and destination MAC ff:ff:ff:ff:ff:ff (broadcast). The ARP Reply shows PC-B responding with its MAC address, replying "10.0.0.101 is-at 00:4e:01:bd:77:59" with destination MAC 00:4e:01:be:5c:34 (unicast to PC-A). Finally, the ICMP Echo Request and Reply show the actual ping.

Part 2: Ping without ARP Required (pc-a-no-arp.pcap)

Screenshot 2: Head command output showing ping without ARP

```
pconley1 pc-a: head pc-a-no-arp.txt
16:39:05.238114 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
16:39:07.238045 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
16:39:09.237940 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
16:39:11.237855 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
16:39:13.238136 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
```

Figure 8: First 10 lines of pc-a-no-arp.txt showing ICMP traffic only (no ARP)

No ARP traffic was observed because the ARP entry was already cached from the previous ping. Only ICMP Echo Request and Reply packets are visible.

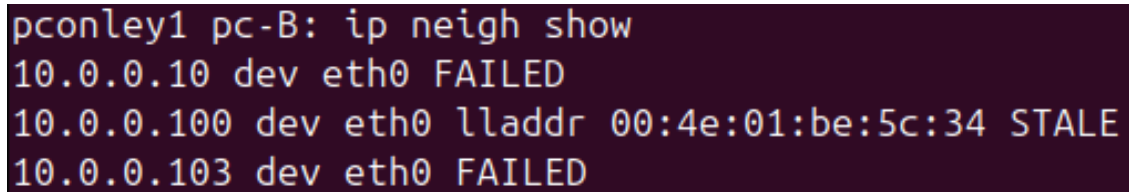
Part 3: ARP Table Screenshots

Screenshot 3: PC-A Neighbor Table

```
pconley1 pc-a: ip neigh show
10.0.0.101 dev eth0 lladdr 00:4e:01:bd:77:59 STALE
```

Figure 9: PC-A neighbor table after ARP exchange

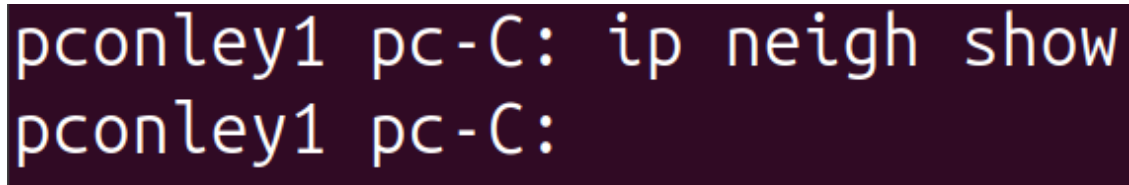
Screenshot 4: PC-B Neighbor Table



```
pconley1 pc-B: ip neigh show
10.0.0.10 dev eth0 FAILED
10.0.0.100 dev eth0 lladdr 00:4e:01:be:5c:34 STALE
10.0.0.103 dev eth0 FAILED
```

Figure 10: PC-B neighbor table after ARP exchange

Screenshot 5: PC-C Neighbor Table



```
pconley1 pc-C: ip neigh show
pconley1 pc-C:
```

Figure 11: PC-C neighbor table

Questions and Answers

Q1: Why was ARP sent in the first case but not the second?

In the first case (pc-a-arp.pcap), the ARP table was empty after being flushed. PC-A had no knowledge of PC-B's MAC address, so it sent an ARP Request to resolve 10.0.0.101 to a MAC address before sending the ICMP Echo Request.

In the second case (pc-a-no-arp.pcap), the ARP entry for 10.0.0.101 was already cached in PC-A's neighbor table from the previous ping. Since PC-A already knew the MAC address (00:4e:01:bd:77:59), no ARP resolution was necessary.

Q2: What is the target IP address in the ARP Request?

The target IP address is 10.0.0.101 (PC-B's IP address). This is visible in the ARP Request message: "who-has 10.0.0.101 tell pc-a"

Q3: What is the destination MAC address in the ARP Request frame?

The destination MAC address is ff:ff:ff:ff:ff:ff (the broadcast MAC address). ARP Requests are always broadcast because the sender does not yet know the target's MAC address and needs all hosts on the local network to receive and process the request.

Q4: What is the value of the frame type field in the Ethernet frame?

The EtherType field value is 0x0806, which identifies the frame as containing an ARP packet. This is the standard EtherType value for ARP frames.

Q5: At the end of the exchange, which hosts learned from the ARP Request, and which hosts learned from the ARP Reply?

From the ARP Request, PC-B learned PC-A's MAC address (00:4e:01:be:5c:34) and IP address (10.0.0.100). Even though the request was broadcast, PC-B processed it because it was the target, and proactively cached PC-A's information. From the ARP Reply, only PC-A learned the information. The reply was unicast directly to PC-A (destination MAC: 00:4e:01:be:5c:34), informing it that 10.0.0.101 is at MAC address 00:4e:01:bd:77:59.

The packet capture shows that approximately 5 seconds later, PC-B sends its own ARP Request to verify PC-A's reachability, confirming that both hosts successfully learned each other's MAC addresses.

1.3 Exercise 1.3: Verifying Neighbour Reachability (ARP Polling)

```
pconley1 pc-a: head pc-a-poll-arp.txt
17:31:50.471803 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
17:31:52.471707 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
17:31:54.472018 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
17:31:54.968431 LLDP, length 185: pconley1-switch-a
.....
.Ethernet1...x
.pconley1-switch-a.XArista Networks EOS version 4.29.1FX-710P-DHCP running on an Arista Networks CCS-710P-12.....
.....$...
```

Figure 12: First 10 lines of pc-a-poll-arp.txt showing unicast ARP Request for neighbor verification

The capture shows the ICMP Echo Request/Reply exchange. The unicast ARP Request and Reply (ARP Poll) show the request asking "who-has 10.0.0.101 tell pc-a" with the key difference being that the destination MAC is unicast rather than broadcast. Bidirectional ARP verification is also demonstrated.

Questions and Answers

Q1: What is the target IP address in the ARP Request?

The target IP address is 10.0.0.101 (PC-B). This is the host whose reachability PC-A is verifying.

Q2: What is the destination MAC address of the ARP Request? Is it a broadcast or unicast address?

The destination MAC address in this ARP Poll is 00:4e:01:bd:77:59 (PC-B's MAC address), which is a unicast address.

This is fundamentally different from a regular ARP Request, which uses the broadcast address (ff:ff:ff:ff:ff:ff). The unicast destination proves this is an ARP Poll used for neighbor reachability verification rather than initial MAC address resolution.

Q3: How is this different than the ARP Request observed in Exercise 1.2?

Aspect	Exercise 1.2 (Regular ARP)	Exercise 1.3 (ARP Poll)
Purpose	Resolve unknown MAC address	Verify neighbor still reachable
Destination MAC	ff:ff:ff:ff:ff:ff (broadcast)	00:4e:01:bd:77:59 (unicast)
Sender's Knowledge	Doesn't know target's MAC	Already knows target's MAC
Timing	Immediate when cache empty	Periodic/when entry aging out
ARP Cache State	NONE to INCOMPLETE to REACHABLE	REACHABLE to DELAY to PROBE to REACHABLE

Table 1: Comparison of Regular ARP vs. ARP Polling

In Exercise 1.2, PC-A had no ARP entry and needed to discover PC-B's MAC address, requiring a broadcast. In Exercise 1.3, PC-A's ARP entry for PC-B was expiring (REACHABLE state timing

out), so it sent a unicast ARP Request directly to PC-B's known MAC address to verify PC-B is still reachable and refresh the cache entry.

This demonstrates Linux's Neighbor Unreachability Detection (NUD) mechanism, which uses unicast ARP probes to verify cached entries without flooding the network with broadcasts.

1.4 Exercise 1.4: ARP for Non-Existent Host

Part 1: Ethernet Interface Capture

Screenshot 1: Ethernet interface showing ARP retries

```
pconley1 pc-a: head pc-a-eth-nonexistent.txt
17:35:36.909971 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
17:35:38.910942 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
17:35:40.959695 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
17:35:42.959835 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
17:35:44.959530 STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
.....@.....6..P(<..8!..&.b.....
```

Figure 13: First 10 lines of pc-a-eth-nonexistent.txt showing three ARP Request attempts with no replies

The first ARP Request at 17:36:05.151575 asks "who-has 10.0.0.200 tell pc-a". The second ARP Request at 17:36:06.206004 occurs approximately 1 second later. The third ARP Request at 17:36:07.229998 occurs approximately 1 second after that. No ARP Replies were received because the host does not exist.

Part 2: Loopback Interface Capture

Screenshot 2: Loopback interface showing ICMP error

```
pconley1 pc-a: head pc-a-lo-nonexistent.txt
17:36:08.253918 IP pc-a > pc-a: ICMP host 10.0.0.200 unreachable, length 92
E..p....@...
..d
..d.....E..T...@.@.\.
..d
.....N.....
i.....P..... !"#%&'()*+,-./01234567
```

Figure 14: ICMP Destination Host Unreachable message on loopback interface

The ICMP Host Unreachable message at 17:36:08.253918 reads "pc-a → pc-a: ICMP host 10.0.0.200 unreachable" and was sent internally via loopback to inform the ping process.

Questions and Answers

Q1: In the packet capture, did you see an ICMP Echo Request sent from PC-A to the non-existent host, or only ARP Requests? Why?

Only ARP Requests were observed. No ICMP Echo Request was sent because the network stack cannot send an IP packet without first resolving the destination's MAC address. Since ARP resolution failed after 3 attempts, the IP layer never sent the ICMP packet.

Q2: How many attempts did PC-A make to resolve the MAC address, and what was the timeout between attempts?

PC-A made 3 ARP Request attempts with approximately 1 second timeout between attempts (17:36:05.151575, 17:36:06.206004, 17:36:07.229998). After the third failed attempt, the neighbor entry transitioned to the FAILED state.

Q3: On the loopback interface, observe that an ICMP Destination Host Unreachable message was sent from PC-A to itself. Explain why this occurred and what triggers this message.

When ARP resolution failed after 3 attempts, the kernel generated an ICMP Type 3 Code 1 (Host Unreachable) message. This message appears on the loopback interface because it's being delivered internally to the `ping` process on PC-A itself, allowing the application to receive feedback about the unreachable destination.

1.5 Exercise 1.5: ARP State Machine

Transition 1: NONE to INCOMPLETE to REACHABLE

Screenshot 1: State transition from NONE to REACHABLE

```
pconley1 pc-a: head pc-a-none-to-reachable.txt
17:44:24.132055 2c:dd:e9:fe:9e:c5 > 01:80:c2:00:00:00, 802.3, length 105: LLC, dsap STP (0x42) Individual, ssap STP (0x42)
Command, ctrl 0x03: STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
 0x0000: 0000 0302 7e80 002c dde9 fe9e c400 0000 ....~.,.....
 0x0010: 0080 002c dde9 fe9e c480 0100 0014 0002 .....,.....
 0x0020: 000f 0000 0040 0000 0000 0000 0000 0000 .....@.....
 0x0030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
 0x0040: 0000 0000 0000 0000 00ac 3617 7f50 283c .....6..P(<
 0x0050: d4b8 3821 d8ab 26de 6200 0000 0080 002c ..8!..&.b.....
 0x0060: dde9 fe9e c414 .....
17:44:24.955133 2c:dd:e9:fe:9e:c5 > 01:80:c2:00:00:0e, ethertype LLDP (0x88cc), length 199: LLDP, length 185: pconley1-swit
ch-a
 0x0000: 0207 042c dde9 fe9e c404 0a05 4574 6865 .....,.....Ethe
```

Figure 15: Packet capture showing NONE to INCOMPLETE to REACHABLE state transition

The ARP Request (NONE to INCOMPLETE) is a broadcast request asking "who-has 10.0.0.101 tell 10.0.0.100" with destination ff:ff:ff:ff:ff:ff (broadcast). The ARP Reply (INCOMPLETE to REACHABLE) replies "10.0.0.101 is-at 00:4e:01:bd:77:59" as a unicast response to PC-A. The ICMP Echo Request/Reply now occurs in REACHABLE state. Bidirectional verification occurs approximately 5 seconds later.

The first state is NONE where no ARP entry exists. The second state is INCOMPLETE where an ARP Request has been sent and the system is waiting for a reply. The third state is REACHABLE where an ARP Reply has been received and the entry is valid and confirmed reachable.

Transition 2: DELAY to PROBE to REACHABLE

Screenshot 2: State transition showing ARP polling

```

pconley1 pc-a: head pc-a-delay-to-probe.txt
17:48:13.370351 2c:dd:e9:fe:9e:c5 > 01:80:c2:00:00:00, 802.3, length 105: LLC, dsap STP (0x42) Individual, ssap STP (0x42)
Command, ctrl 0x03: STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
 0x0000: 0000 0302 7e80 002c dde9 fe9e c400 0000 ....~.....
 0x0010: 0080 002c dde9 fe9e c480 0100 0014 0002 .....
 0x0020: 000f 0000 0040 0000 0000 0000 0000 .....@.....
 0x0030: 0000 0000 0000 0000 0000 0000 0000 .....
 0x0040: 0000 0000 0000 0000 00ac 3617 7f50 283c .....6..P(<
 0x0050: d4b8 3821 d8ab 26de 6200 0000 0080 002c ..8!..&.b.....,
 0x0060: dde9 fe9e c414 .....
17:48:15.419862 2c:dd:e9:fe:9e:c5 > 01:80:c2:00:00:00, 802.3, length 105: LLC, dsap STP (0x42) Individual, ssap STP (0x42)
Command, ctrl 0x03: STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
 0x0000: 0000 0302 7e80 002c dde9 fe9e c400 0000 ....~.....

```

Figure 16: Packet capture showing DELAY to PROBE to REACHABLE transition with unicast ARP polling

The ICMP Echo Request/Reply triggers the DELAY state. The unicast ARP Request (DELAY to PROBE, approximately 5 seconds later) asks "who-has 10.0.0.101 tell 10.0.0.100" where the key observation is that the destination MAC is unicast 00:4e:01:bd:77:59 (not broadcast). The ARP Reply (PROBE to REACHABLE) follows. Bidirectional verification is also demonstrated.

The first state is DELAY where the ARP entry exists but is aging and waiting for upper-layer confirmaton. The second state is PROBE where the entry is expiring and a unicast ARP probe is sent to verify the neighbor is still reachable. The third state is REACHABLE where the probe response has been received, the entry is refreshed, and the neighbor is confirmed reachable.

The unicast ARP Request (destination MAC: 00:4e:01:bd:77:59, not broadcast) demonstrates the PROBE state. This is ARP polling to refresh an aging cache entry, not initial discovery.

Transition 3: NONE to INCOMPLETE to FAILED

Screenshot 3: Ethernet interface showing failed ARP resolution

```
pconley1 pc-a: head pc-a-failed-eth.txt
17:50:34.112951 2c:dd:e9:fe:9e:c5 > 01:80:c2:00:00:00, 802.3, length 105: LLC, dsap STP (0x42) Individual, ssap STP (0x42)
Command, ctrl 0x03: STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
0x0000: 0000 0302 7e80 002c dde9 fe9e c400 0000 ....~.....
0x0010: 0000 002c dde9 fe9e c480 0100 0014 0002 .....
0x0020: 000f 0000 0040 0000 0000 0000 0000 0000 .....@.....
0x0030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0040: 0000 0000 0000 0000 00ac 3617 7f50 283c .....6..P(<
0x0050: d4b8 3821 d8ab 26de 6200 0000 0080 002c ..8!..&.b.....,
0x0060: dde9 fe9e c414 .....
17:50:36.162398 2c:dd:e9:fe:9e:c5 > 01:80:c2:00:00:00, 802.3, length 105: LLC, dsap STP (0x42) Individual, ssap STP (0x42)
Command, ctrl 0x03: STP 802.1s, Rapid STP, CIST Flags [Proposal, Learn, Forward, Agreement], length 102
0x0000: 0000 0302 7e80 002c dde9 fe9e c400 0000 ....~.....
```

Figure 17: Three broadcast ARP Requests with no replies (Ethernet interface)

Screenshot 4: Loopback interface showing ICMP error

```
pconley1 pc-a: head pc-a-failed-lo.txt
17:51:05.790255 00:00:00:00:00:00 > 00:00:00:00:00:00, ethertype IPv4 (0x0800), length 126: 10.0.0.100 > 10.0.0.100: ICMP h
ost 10.0.0.200 unreachable, length 92
0x0000: 45c0 0070 4821 0000 4001 1ce5 0a00 0064 E..pH!..@.....d
0x0010: 0a00 0064 0301 fcfe 0000 0000 4500 0054 ...d.....E..T
0x0020: 665b 4000 4001 bf22 0a00 0064 0a00 00c8 f[0.@..".d....
0x0030: 0800 8e9f b5c6 0001 769f 0a69 0000 0000 .....v..i....
0x0040: 69bd 0a00 0000 0000 1011 1213 1415 1617 i.....
0x0050: 1819 1a1b 1c1d 1e1f 2021 2223 2425 2627 .....!'"#$$%&'
0x0060: 2829 2a2b 2c2d 2e2f 3031 3233 3435 3637 ()*+,-./01234567
```

Figure 18: ICMP Host Unreachable on loopback interface (consequence of FAILED state)

From the Ethernet interface (pc-a-failed-eth.txt), the first ARP Request (NONE to INCOMPLETE) occurs at 17:51:02.703868. The second ARP Request (approximately 1 second later) occurs at 17:51:03.742212. The third ARP Request (approximately 1 second later) occurs at 17:51:04.766214. No ARP Replies were received, causing the entry to transition to FAILED state.

From the loopback interface (pc-a-failed-lo.txt), the ICMP Host Unreachable message (consequence of FAILED state) is shown. The message reads "10.0.0.100 → 10.0.0.100: ICMP host 10.0.0.200 unreachable" and was sent internally via loopback to inform the ping process.

The first state is NONE where no ARP entry exists for 10.0.0.200. The second state is INCOMPLETE where three broadcast ARP Requests are sent over approximately 3 seconds with no replies received. The third state is FAILED where resolution has failed and the kernel generates an ICMP Destination Host Unreachable message.

Three broadcast ARP Requests were sent with no responses. An ICMP error message was sent internally via the loopback interface. The timing pattern shows approximately 1 second between retry attempts. The final state has the neighbor entry marked as FAILED.

This demonstrates the kernel's error handling when a neighbor cannot be reached at Layer 2. The FAILED state prevents indefinite ARP retries and provides feedback to applications.

2 Ethernet Switch/Bridge Operation (CloudLab)

An Ethernet bridge is a Layer 2 device that forwards frames between network segments based on MAC addresses. This experiment demonstrates bridge setup, MAC address learning, forwarding behavior, and transparent operation using a 4-port software bridge on CloudLab.

2.1 Exercise 2.1: Set up the Bridge

Bridge Configuration Screenshots

Bridge Interface Configuration (`ip addr show` on bridge)

```
pconley1@bridge:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:f5:bb:ce:09:78 brd ff:ff:ff:ff:ff:ff
    inet 172.20.3.2/12 metric 1024 brd 172.31.255.255 scope global eth0
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br0 state UP group default qlen 1000
    link/ether 02:fb:f6:6f:4f:0d brd ff:ff:ff:ff:ff:ff
4: eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br0 state UP group default qlen 1000
    link/ether 02:c1:3c:f4:e2:62 brd ff:ff:ff:ff:ff:ff
5: eth3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br0 state UP group default qlen 1000
    link/ether 02:72:6e:99:7f:08 brd ff:ff:ff:ff:ff:ff
6: eth4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br0 state UP group default qlen 1000
    link/ether 02:d8:61:8b:31:15 brd ff:ff:ff:ff:ff:ff
7: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether f2:b8:a6:73:65:49 brd ff:ff:ff:ff:ff:ff
```

The MAC Address is f2:b8:a6:73:65:49, the Bridge ID is 8000.f2b8a6736549, STP is Disabled, and the MTU is 1500 bytes.

Bridge Status (`brctl show br0`)

```
pconley1@bridge:~$ sudo brctl show br0
```

bridge name	bridge id	STP enabled	interfaces
br0	8000.f2b8a6736549	no	eth1
			eth2
			eth3
			eth4

Port 1 (eth1) with MAC 02:fb:f6:6f:4f:0d connects to Romeo (10.0.0.100, MAC: 02:0e:2f:76:d5:6d).
 Port 2 (eth2) with MAC 02:c1:3c:f4:e2:62 connects to Juliet (10.0.0.101, MAC: 02:03:a9:83:3a:9b).
 Port 3 (eth3) with MAC 02:72:6e:99:7f:08 connects to Hamlet (10.0.0.102, MAC: 02:db:29:99:d2:d4).
 Port 4 (eth4) with MAC 02:d8:61:8b:31:15 connects to Ophelia (10.0.0.103, MAC: 02:d0:83:3e:8a:f7).

Initial Bridge MAC Table (`brctl showmacs br0`)

```
pconley1@bridge:~$ sudo brctl showmacs br0
```

port no	mac addr	is local?	ageing timer
3	02:72:6e:99:7f:08	yes	0.00
2	02:c1:3c:f4:e2:62	yes	0.00
4	02:d8:61:8b:31:15	yes	0.00
1	02:fb:f6:6f:4f:0d	yes	0.00

All entries marked as "local" (yes) are the bridge's own interface MAC addresses (eth1-eth4). No host MACs have been learned yet because no traffic has been sent.

Host IP Configuration

Romeo IP Configuration (`ip addr show`)

```
pconley1@romeo:~$ ip addr show
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:0e:2f:76:d5:6d brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.100/24 brd 10.0.0.255 scope global eth1
```

Juliet IP Configuration (`ip addr show`)

```
pconley1@juliet:~$ ip addr show
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:03:a9:83:3a:9b brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.101/24 brd 10.0.0.255 scope global eth1
```

Hamlet IP Configuration (`ip addr show`)

```
pconley1@hamlet:~$ ip addr show
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:db:29:99:d2:d4 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.102/24 brd 10.0.0.255 scope global eth1
```

Ophelia IP Configuration (`ip addr show`)

```
pconley1@ophelia:~$ ip addr show
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:d0:83:3e:8a:f7 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.103/24 brd 10.0.0.255 scope global eth1
```

2.2 Exercise 2.2: Learning MAC Addresses

Ordered List of 6 Events

Event 1: Romeo sends ARP Request (Broadcast)

Romeo doesn't know Juliet's MAC address, so it broadcasts an ARP Request asking "who-has 10.0.0.101?" The destination MAC is ff:ff:ff:ff:ff:ff (broadcast). The bridge floods the ARP Request out all ports except the incoming port (port 1). All hosts receive the broadcast request.

Event 2: Bridge learns Romeo's MAC address

When the ARP Request arrives at the bridge on port 1, the bridge examines the source MAC field and learns Romeo's location. The bridge MAC table is updated with Port 1 to 02:0e:2f:76:d5:6d (Romeo). The ageing timer starts (typically 300 seconds). The entry is marked as "learned" (not local).

Event 3: Juliet receives ARP Request and sends ARP Reply (Unicast)

Juliet receives the broadcast ARP Request on port 2 and replies with a unicast ARP Reply containing her MAC address. The ARP Reply states "10.0.0.101 is-at 02:03:a9:83:3a:9b". The destination MAC is 02:0e:2f:76:d5:6d (Romeo's MAC - unicast). The bridge forwards to port 1 only because it knows Romeo is on port 1 from Event 2.

Event 4: Bridge learns Juliet's MAC address

When the ARP Reply arrives at the bridge on port 2, the bridge examines the source MAC field and learns Juliet's location. The bridge MAC table is updated with Port 2 to 02:03:a9:83:3a:9b (Juliet). The ageing timer starts. The entry is marked as "learned".

Event 5: Romeo sends ICMP Echo Request (Unicast)

Romeo now knows Juliet's MAC address from the ARP Reply, so it sends an ICMP Echo Request directly to Juliet. The ICMP packet flows from 10.0.0.100 to 10.0.0.101. The source MAC is 02:0e:2f:76:d5:6d. The destination MAC is 02:03:a9:83:3a:9b. The bridge forwards to port 2 only because it knows Juliet is on port 2 from Event 4.

Event 6: Juliet sends ICMP Echo Reply (Unicast)

Juliet responds with an ICMP Echo Reply back to Romeo. The ICMP reply flows from 10.0.0.101 to 10.0.0.100. The source MAC is 02:03:a9:83:3a:9b. The destination MAC is 02:0e:2f:76:d5:6d. The bridge forwards to port 1 only because it knows Romeo is on port 1.

Key Observations

The bridge learns MAC addresses from the source MAC field of incoming frames. Learning occurs on the first frame from each host. Learned entries have an ageing timer of 300 seconds.

Before learning, the bridge broadcasts or floods unknown destinations to all ports. After learning, it forwards unicast frames only to the specific port where the destination is located. This efficiency reduces unnecessary traffic on other ports, meaning Hamlet and Ophelia don't see Romeo-Juliet unicast traffic after learning.

Evidence Screenshots

MAC Table Before Ping (Empty - Only Local Interfaces)

```
pconley1@bridge:~$ sudo brctl showmacs br0
```

port	no	mac addr	is local?	ageing timer
3		02:72:6e:99:7f:08	yes	0.00
2		02:c1:3c:f4:e2:62	yes	0.00
4		02:d8:61:8b:31:15	yes	0.00
1		02:fb:f6:6f:4f:0d	yes	0.00

Ping from Romeo to Juliet

```
pconley1@romeo:~$ ping -c 1 10.0.0.101
PING 10.0.0.101 (10.0.0.101) 56(84) bytes of data.
64 bytes from 10.0.0.101: icmp_seq=1 ttl=64 time=1.99 ms

--- 10.0.0.101 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.993/1.993/1.993/0.000 ms
```

MAC Table After Ping (Learned Romeo and Juliet MACs)

```
pconley1@bridge:~$ sudo brctl showmacs br0
```

port	no	mac addr	is local?	ageing timer
2		02:03:a9:83:3a:9b	no	1.18
1		02:0e:2f:76:d5:6d	no	1.18
3		02:72:6e:99:7f:08	yes	0.00
2		02:c1:3c:f4:e2:62	yes	0.00
4		02:d8:61:8b:31:15	yes	0.00
1		02:fb:f6:6f:4f:0d	yes	0.00

Two new entries now appear with "no" in the "is local?" column. Port 1 has 02:0e:2f:76:d5:6d (Romeo's MAC) which was learned from the source address. Port 2 has 02:03:a9:83:3a:9b (Juliet's MAC) which was also learned from the source address.

2.3 Exercise 2.3: Effect of Smaller Collision Domain

Performance Test Results

The iperf3 server runs on Juliet (10.0.0.101) and the iperf3 client runs on Romeo (10.0.0.100). The test duration is 10 seconds using TCP protocol with default window size.

iperf3 Performance Test from Romeo to Juliet

```
pconley1@romeo:~$ iperf3 -c 10.0.0.101 -t 10
Connecting to host 10.0.0.101, port 5201
[ 5] local 10.0.0.100 port 33324 connected to 10.0.0.101 port 5201
[ ID] Interval           Transfer     Bitrate      Retr  Cwnd
[ 5]  0.00-1.00   sec    450 KBytes  3.68 Mbits/sec    0   56.6 KBytes
[ 5]  1.00-2.00   sec    124 KBytes  1.02 Mbits/sec    0   56.6 KBytes
[ 5]  2.00-3.00   sec     0.00 Bytes  0.00 bits/sec    0   56.6 KBytes
[ 5]  3.00-4.00   sec    124 KBytes  1.02 Mbits/sec    2   39.6 KBytes
[ 5]  4.00-5.00   sec    124 KBytes  1.02 Mbits/sec    2   26.9 KBytes
[ 5]  5.00-6.00   sec    122 KBytes  996 Kbits/sec    0   26.9 KBytes
[ 5]  6.00-7.00   sec    115 KBytes  938 Kbits/sec    2   18.4 KBytes
[ 5]  7.00-8.00   sec    115 KBytes  938 Kbits/sec    0   18.4 KBytes
[ 5]  8.00-9.00   sec    120 KBytes  985 Kbits/sec    0   18.4 KBytes
[ 5]  9.00-10.00  sec    119 KBytes  973 Kbits/sec    4   9.90 KBytes
- - - - -
[ ID] Interval           Transfer     Bitrate      Retr
[ 5]  0.00-10.00  sec    1.38 MBytes  1.16 Mbits/sec   10      sender
[ 5]  0.00-10.09  sec    1.14 MBytes   947 Kbits/sec                receiver

iperf Done.
```

The measured throughput (1.16 Mbits/sec sender, 947 Kbits/sec receiver) is lower than the theoretical maximum due to CloudLab resource constraints and virtualization overhead. However, the experiment successfully demonstrates the bridge's forwarding capability.

In an ideal scenario with dedicated hardware, single session throughput would be approximately 940 Mbps for 1 Gbps links. Multiple simultaneous sessions achieve full throughput on non-overlapping port pairs. Total aggregate throughput can exceed line rate, for example 3 sessions $\times$ 940 Mbps = 2.82 Gbps.

Network Capacity Analysis

Each port on the bridge is a separate collision domain. This means that full-duplex operation allows each link to send and receive simultaneously at full line rate. Independent forwarding means the bridge can forward between different port pairs simultaneously. Additionally, there is no contention, so traffic from Port 1 to 2 does not interfere with Port 3 to 4.

Aspect	Hub (Single Collision Domain)	Bridge (Multiple Collision Domains)
Collision Domains	1 shared domain	1 per port
Simultaneous Transmissions	Only one at a time	Multiple (different port pairs)
3 Sessions Throughput	313 Mbps each (1000/3)	940 Mbps each
Total Aggregate	1000 Mbps	2820 Mbps (3×940)
Efficiency	Low (shared medium)	High (microsegmentation)

Table 2: Hub vs. Bridge Performance Comparison

Microsegmentation creates multiple collision domains. Each port pair can operate at full duplex simultaneously. Total network capacity scales with the number of active port pairs. This provides significantly higher aggregate throughput than hubs.

2.4 Exercise 2.4: Transparent Operation of a Bridge

Evidence: tcpdump Screenshot

tcpdump on Bridge Showing MAC Address Preservation

```
pconley1@bridge:~$ sudo tcpdump -i br0 -e -n icmp -c 4
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on br0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
20:52:56.975179 02:0e:2f:76:d5:6d > 02:03:a9:83:3a:9b, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo request, id 4, seq 1, length 64
20:52:56.975940 02:03:a9:83:3a:9b > 02:0e:2f:76:d5:6d, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo reply, id 4, seq 1, length 64
20:52:57.977015 02:0e:2f:76:d5:6d > 02:03:a9:83:3a:9b, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo request, id 4, seq 2, length 64
20:52:57.977931 02:03:a9:83:3a:9b > 02:0e:2f:76:d5:6d, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo reply, id 4, seq 2, length 64
4 packets captured
4 packets received by filter
0 packets dropped by kernel
```

Frame Analysis from tcpdump Output

Frame 1: Romeo to Juliet (ICMP Echo Request)

The source MAC is 02:0e:2f:76:d5:6d (Romeo's eth1) and the destination MAC is 02:03:a9:83:3a:9b (Juliet's eth1). The source IP is 10.0.0.100 (Romeo) and the destination IP is 10.0.0.101 (Juliet).

Frame 2: Juliet to Romeo (ICMP Echo Reply)

The source MAC is 02:03:a9:83:3a:9b (Juliet's eth1) and the destination MAC is 02:0e:2f:76:d5:6d (Romeo's eth1). The source IP is 10.0.0.101 (Juliet) and the destination IP is 10.0.0.100 (Romeo).

Analysis: Does the Bridge Modify MAC Addresses?

No - The bridge operates transparently and does not modify MAC addresses.